AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH

CareLink

Autor:
Wiktor Matusik
Mateusz Kowalczyk

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

1.1. Cel główny projektu 4
1.2. Zakres funkcjonalny programu (Co aplikacja musi robić) 4
1.3. Wymagania niefunkcjonalne 5
1.4. Koncept wyglądu aplikacji 5

2. Określenie wymagań szczegółowych **6**

2.1. Cel i zakres 6
2.2. Dane wejściowe, interfejs użytkownika i układ 6
2.3. Zachowanie aplikacji i zdarzenia 7
2.4. Zachowania aplikacji w sytuacjach niepożądanych 8
2.5. Możliwości dalszego rozwoju oprogramowania 9

3. Projektowanie **10**

3.1. Opis przygotowania narzędzi 10
3.2. Wybór bibliotek, struktura klas i serwisów 10
3.3. Implementacja interfejsu użytkownika i dodawanie elementów sterujących 11
3.4. Struktura layoutów i rozmieszczenie elementów 13
3.5. Dokładne określenie założeń i działania ekranów 14
3.6. Opis wykorzystanych algorytmów i logiki urządzeń 16

4. Implementacja **17**

4.1. Opis nowo utworzonych plików za co odpowiadają 17
4.2. Szkielet kodu, struktury i zmienne 18
4.3. Opis dziedziczenia w architekturze aplikacji 19
4.4. Opis procedur, metod i ich działania 19

5. Testowanie **22**

6. Podręcznik użytkownika **23**

Literatura **24**

Spis rysunków **24**

Spis tabel **25**

Spis listingów **26**

1. Ogólne określenie wymagań

1.1. Cel główny projektu

Celem projektu jest stworzenie aplikacji mobilnej CareLink, która służy do zdalnego wsparcia i monitorowania bezpieczeństwa osób starszych przez ich opiekunów. Aplikacja ma ułatwiać codzienne spacery seniorów, dbać o ich bezpieczeństwo za pomocą czujników w smartfonie oraz zapewniać opiekunom stały wgląd w lokalizację i stan podopiecznego.

1.2. Zakres funkcjonalny programu (Co aplikacja musi robić)

Wymagania funkcjonalne definiują kluczowe możliwości oraz moduły, jakie system musi bezwzględnie realizować. Zakres ten obejmuje następujące elementy:

- System kont i ról użytkowników:
  - Wspólny ekran logowania dla wszystkich użytkowników.
  - Formularz rejestracji z możliwością wyboru typu konta: Senior, Opiekun lub Administrator.
  - Automatyczne dopasowanie widoku i uprawnień aplikacji w zależności od zalogowanego typu konta.
- Moduł Seniora (uproszczony interfejs):
  - Duży, łatwo dostępny przycisk alarmowy SOS do natychmiastowego wzywania pomocy.
  - Kompas domowy: funkcja wskazująca północ lub wyznaczająca kierunek powrotu do wcześniej zapisanego punktu (domu), przydatna w razie utraty orientacji w terenie.
  - Licznik dystansu spaceru (ile kilometrów pokuto danego dnia).
- Moduł Opiekuna (panel monitorowania):
  - Podgląd trasy spaceru seniora na mapie (z zaznaczonym przebytym dystansem w kilometrach).
  - System inteligentnych powiadomień i alarmów reagujących na:
    - Akcelerometr / Żyroskop: wykrycie nagłego upadku lub dłuższego braku ruchu (stania w miejscu).

    * Czujnik światła: wykrywanie poruszania się w nocy (wysłanie ostrzeżenia do opiekuna o nocnym spacerze).

1.3. Wymagania niefunkcjonalne

Aspekty niefunkcjonalne określają cechy jakościowe, technologiczne oraz standardy, jakim system musi odpowiadać. W tej grupie wyróżnia się następujące wytyczne:

- Dostępność (Accessibility): Interfejs dla seniora musi być maksymalnie uproszczony, czytelny i posiadać duże elementy dotykowe, aby obsługa była bezproblemowa.
- Technologia: Aplikacja napisana w technologii wieloplatformowej (.NET MAUI), co zapewnia stabilność i możliwość uruchomienia na różnych urządzeniach mobilnych.

1.4. Koncept wyglądu aplikacji

Konstrukcja wizualna interfejsu opiera się na zasadzie przejrzystości oraz adaptacji struktury widoków do potrzeb i roli zalogowanego użytkownika. Konspekt projektu graficznego obejmuje:

- Wspólna stylistyka i spójność wizualna: Zastosowanie kontrastowej palety barw o wysokim wskaźniku czytelności, ułatwiającej korzystanie z aplikacji osobom ze słabszym wzrokiem.
- Layout interfejsu Seniora: Zorientowany pionowo, jednogramaturowy układ z dominantą wizualną w postaci centralnie umieszczonego przycisku SOS oraz dolną nawigacją kaflową o dużym obszarze klikalnym.
- Layout interfejsu Opiekuna: Układ pulpitowy z dominantą interaktywnej mapy oraz dolnym lub bocznym panelem informacyjnym wyświetlającym powiadomienia i statystyki.
- Layout interfejsu Administratora: Przejrzysty panel z tabelami danych, wskaźnikami statusu oraz formularzami zarządzania powiązaniami ról.

2. Określenie wymagań szczegółowych

2.1. Cel i zakres

Precyzyjne zdefiniowanie granic projektu pozwala na dokładne zrozumienie architektury biznesowej i technicznej aplikacji.

- Cel główny

  Celem aplikacji CareLink jest zapewnienie wszechstronnego wsparcia bezpieczeństwa osobom starszym oraz zagwarantowanie spokoju ich opiekunom. System integruje technologie mobilne (GPS, czujniki fizyczne urządzenia) z intuicyjnym interfejsem dostosowanym do potrzeb seniorów, umożliwiając monitorowanie lokalizacji, aktywności fizycznej oraz natychmiastowe reagowanie w sytuacjach zagrożenia życia lub zdrowia.

- Zakres funkcjonalny

  Aplikacja została zaprojektowana jako rozwiązanie wieloplatformowe (.NET MAUI) obsługujące trzy odrębne typy kont użytkowników (Senior, Opiekun, Administrator) z poziomu wspólnego punktu wejścia. Zakres obejmuje moduły śledzenia trasy, analizy sensorów (akcelerometr, żyroskop, czujnik światła, magnetometr), powiadomień kryzysowych oraz zarządzania uprawnieniami.

2.2. Dane wejściowe, interfejs użytkownika i układ

Układ graficzny oraz sposób dostarczania danych zostały zaprojektowane z uwzględnieniem specyfiki poszczególnych ról użytkowników. W strukturze interfejsu wyróżniamy:

- Ekran logowania i rejestracji (Wspólny)
  - Dane wejściowe: Adres e-mail, hasło, wybór roli (podczas rejestracji spośród: Senior, Opiekun, Administrator).
  - Wygląd layoutu: Minimalistyczny, wysoce czytelny formularz. Pola tekstowe posiadają duże etykiety oraz opcję odsłonięcia wpisywanego hasła. Przycisk „Zaloguj się” zajmuje pełną szerokość ekranu w dolnej partii formularza.
- Widok dedykowany dla Seniora

  - Dane wejściowe/sterowanie: Ekran dotykowy, czujniki wewnętrzne smartfona (GPS, akcelerometr, magnetometr, czujnik światła).
  - Wygląd layoutu:
  - Górny pasek statusu: Wyraźny wskaźnik stanu baterii oraz informacja o statusie połączenia z opiekunem (np. zielona tarcza „Opiekun czuwa").
  - Centralny element: Ogromny, czerwony przycisk SOS opatrzony tekstem „WEZWIJ POMOC!" oraz instrukcją „Przytrzymaj 3 sekundy".
  - Dolny panel (Moje Narzędzia): Duże, kontrastowe kafelki dotykowe:
    * Kompas domowy: Ikona kompasu i domu.
    * Mój Spacer: Licznik pokonanego dystansu w kilometrach.
    * Kontakt: Przycisk szybkiego połączenia z opiekunem.

- Widok dedykowany dla Opiekuna
  - Dane wejściowe: Dane strumieniowe GPS i statusy sensorów nadsyłane z urządzenia podopiecznego.
  - Wygląd layoutu:
    * Górny pasek: Wybór podopiecznego oraz globalny status bezpieczeństwa.
    * Obszar mapy: Interaktywna mapa zajmująca 60–70% powierzchni ekranu, prezentująca bieżącą lokalizację seniora oraz niebieską linię przebytej trasy (wraz z podsumowaniem kilometrów).
    * Panel powiadomień: Kafelki alertów informujące o nocnych spacerach, upadkach lub bezczynności.

- Widok dedykowany dla Administratora
  - Wygląd layoutu: Panel analityczno-zarządczy. Zawiera zestawienia statystyczne (liczba aktywnych seniorów, liczba alarmów SOS) oraz tabele parowania kont (przypisanie opiekuna do seniora).

2.3. Zachowanie aplikacji i zdarzenia

Reakcje systemu na działania użytkownika oraz autonomiczne procesy w tle decydują o niezawodności oprogramowania. Zasady te dzielą się na:

- Interakcje użytkownika (Reakcje na kliknięcie)

  - Przycisk SOS (Senior): Wymaga przytrzymania przez 3 sekundy (zapobiega przypadkowym kliknięciom). Po zweryfikowaniu przytrzymania aplikacja natychmiast generuje alarm, wysyła powiadomienie PUSH do opiekuna wraz z aktualnymi współrzędnymi GPS.
  - Kompas domowy (Senior): Po kliknięciu kafelka przełącza widok na dedykowany ekran nawigacyjny oferujący dwie opcje: wskazanie północy geograficznej lub dynamiczną strzałkę kierującą w stronę wcześniej zapisanej lokalizacji domowej.
  - Szybki kontakt (Senior): Kliknięcie inicjuje połączenie telefoniczne pod zaprogramowany numer opiekuna.

- Zdarzenia automatyczne (Praca w tle i sensory)
  - Moduł GPS i Trasa: Aplikacja cyfrowo rejestruje współrzędne geograficzne w interwałach czasowych, tworząc polilinię na mapie opiekuna oraz przeliczając przebyty dystans w kilometrach.
  - Akcelerometr i Żyroskop (Wykrywanie upadku/bezruchu): Algorytmy w tle analizują nagłe przeciążenia (wstrząsy) oraz stan, w którym lokalizacja GPS nie zmienia się mimo upływu czasu. W przypadku wykrycia anomalii aplikacja uruchamia procedurę ostrzegawczą (pyta seniora, czy wszystko w porządku, a w razie braku odpowiedzi wysyła alert do opiekuna).
  - Czujnik światła (Zmierzch/Noc): System monitoruje natężenie oświetlenia otoczenia. Jeśli senior porusza się w warunkach nocnych (niskie oświetlenie + wykryty ruch GPS), aplikacja automatycznie generuje powiadomienie dla opiekuna o treści: „Uwaga! Podopieczny porusza się w nocy”.

2.4. Zachowania aplikacji w sytuacjach niepożądanych

Odporność aplikacji na niesprzyjające warunki środowiskowe i błędy użytkownika została zabezpieczona specjalnymi procedurami awaryjnymi. Wdrożone mechanizmy obejmują:

- Brak sygnału GPS (np. między wysokimi budynkami lub w budynkach): Aplikacja automatycznie przełącza tryb orientacji z GPS na kompas oparty na magnetometrze urządzenia, umożliwiając seniorowi odczyt kierunku północy lub powrotu do domu na podstawie ostatnio znanych danych.
- Utrata połączenia internetowego: Dane o lokalizacji oraz zdarzeniach z czujników są buforowane lokalnie w pamięci podręcznej urządzenia. Po ponownym

uzyskaniu połączenia siecowego pakiety danych są automatycznie wysyłane do bazy danych i opiekuna.

- Rozładowanie baterii lub awaria telefonu seniora: System odnotowuje brak pulsacyjnego sygnału (heartbeat) z urządzenia i po upływie określonego czasu wysyła powiadomienie alarmowe do opiekuna o utracie łączności z podopiecznym.

- Błędne dane logowania: Aplikacja wyświetla czytelny, pełnoekranowy komunikat z instrukcją korekty danych, nie powodując zamknięcia ani awarii programu.

2.5. Możliwości dalszego rozwoju oprogramowania

Perspektywy rozbudowy systemu wyznaczają kierunki przyszłych ulepszeń technologicznych i funkcjonalnych. Możliwe modyfikacje obejmują:

- Integracja ze Smartwatchami: Rozbudowa systemu o dedykowaną aplikację na zegarki inteligentne (WearOS / watchOS), co umożliwi odczyt tętna, saturacji oraz ułatwi wykrywanie upadków bezpośrednio z nadgarstka.

- Moduł przypomnień o lekach: Wprowadzenie systemu powiadomień głosowych i tekstowych dla seniora przypominających o konieczności przyjęcia leków o określonych godzinach, wraz z raportem dla opiekuna o zażyciu dawek.++

- Wielojęzyczność i Panel Webowy: Dodanie obsługi wielu języków oraz pełnoprawnej aplikacji webowej dla administratora i ośrodków opiekuńczych zarządzających wieloma podopiecznymi jednocześnie.

3. Projektowanie

3.1. Opis przygotowania narzędzi

Proces przygotowania oprogramowania CareLink wymagał skonfigurowania zintegrowanego środowiska programistycznego oraz systemu kontroli wersji:

- Visual Studio:
  - Wymagana jest instalacja pakietu obciążenia .NET Multi-platform App UI development (.NET MAUI).
  - Należy zainstalować komponenty SDK dla Androida oraz skonfigurować wirtualne urządzenie w Android Device Manager.
  - W przypadku testowania lokalnego na systemie Windows wymagana jest aktywacja trybu deweloperskiego w ustawieniach systemu Windows.
- System kontroli wersji Git & GitHub:
  - Repozytorium projektu zostało skonfigurowane z główną gałęzią master.
  - Zarządzanie kodem w Visual Studio odbywa się poprzez wbudowany panel Git Changes.
  - Standardowy przepływ pracy obejmuje lokalne zatwierdzenie zmian (Commit) oraz ich synchronizację z serwerem zdalnym na GitHubie (Pull przed rozpoczęciem pracy w celu pobrania najnowszej wersji oraz Push po wykonaniu zatwierdzenia).

3.2. Wybór bibliotek, struktura klas i serwisów

Architektura aplikacji *CareLink* została oparta o wzorzec projektowy **MVVM (Model-View-ViewModel)**, wspierany przez pakiet `CommunityToolkit.Mvvm`. Podział klas i serwisów przedstawia się następująco:

- **Zależności i pakiety (NuGet):**
  - `CommunityToolkit.Mvvm`: Omotanie właściwości i komend z wykorzystaniem generatorów kodów źródłowych (np. atrybuty `[ObservableProperty]` i `[RelayCommand]`).
  - `sqlite-net-pcl` oraz `SQLitePCLRaw`: Obsługa lokalnej bazy danych SQLite przechowującej konta użytkowników w pamięci urządzenia.

  - `FirebirdSql.Data.FirebirdClient`: Klient dostępu do zewnętrznej relacyjnej bazy danych Firebird używanej do integracji chmurowej.

- **Modele (`Models`):**
  - `User.cs`: Odpowiada za odwzorowanie tabeli użytkowników w lokalnej bazie danych SQLite. Zawiera właściwości `Id` (klucz główny z auto-inkrementacją), `Username`, `Password` oraz `Role` (rola w systemie: Senior lub Opiekun).

- **Serwisy (`Services`):**
  - `LocalDatabaseService.cs`: Zarządza lokalnym połączeniem z plikiem bazy danych `AplikacjaSeniora.db3`. Odpowiada za asynchroniczną inicjalizację tabeli użytkowników, rejestrację nowych kont z blokadą duplikowania nazw oraz weryfikację poświadczeń podczas logowania.
  - `DatabaseService.cs`: Dedykowana usługa chmurowa realizująca zapytania SQL (`FbConnection`, `FbCommand`) do serwera Firebird przy użyciu parametrów zapobiegających atokom SQL Injection.

- **Logika prezentacji (`PageModels`):**
  - `MainPageViewModel.cs`: Zarządza stanem pulpitu głównego, odczytuje poziom baterii (`Battery.Default`), obsługuje licznik czasu realnego (`IDispatcherTime`
nawigację oraz zdarzenia SOS i powiadomień.
  - `CompassPageViewModel.cs`: Obsługuje czujnik kompasu urządzenia (`Compass.Default`
oblicza kąt nachylenia strzałki nawigacyjnej, przełącza tryby pracy oraz bezpiecznie zatrzymuje czujniki po wyjściu z ekranu.
  - `WalkPageViewModel.cs`: Kontroluje stan śledzenia aktywności (start/stop) oraz statystyki przebytego dystansu i czasu spaceru.

- **Zarządzanie nawigacją (`AppShell.xaml.cs`):**
  - Nawigacja opiera się na Shell Routing opartym na adresach URI. Klasa `AppShell` rejestruje trasy dla poszczególnych widoków: `LoadingPage`, `LoginPage`, `RegisterPage`, `MainPage`, `CompassPage` oraz `WalkPage`.

3.3. Implementacja interfejsu użytkownika i dodawanie elementów sterujących

W technologii .NET MAUI interfejsy widoków są definiowane w plikach języka znaczników XAML (`.xaml`), co pozwala na czyste oddzielenie struktury wizualnej od logiki

aplikacji (MVVM). Poniżej opisano strukturę oraz sposób dodawania podstawowych elementów interfejsu na przykładzie zrealizowanych ekranów **Kompasu** oraz **Spaceru**.

- Układ bazowy (Grid) i podział ekranu Oba ekrany (`CompassPage` oraz `WalkPage`) opierają się na kontrolce `Grid` (Siatka), która dzieli dostępną przestrzeń ekranu na wiersze o różnych właściwościach (`RowDefinitions="Auto, *, Auto"`):
  - **Wiersz 0 (`Auto`):** Górny nagłówek z tytułem oraz statusem działania modułu.
  - **Wiersz 1 (`*`):** Elastyczny obszar środkowy, zajmujący całą pozostałą przestrzeń, w którym umieszczane są główne elementy wizualne (tarcza kompasu lub kafelki ze statystykami).
  - **Wiersz 2 (`Auto`):** Dolny panel z przyciskami nawigacyjnymi i akcji.
- Jak dodać przycisk w XAML? (`Button`)

  Aby dodać interaktywny przycisk, który użytkownik (w tym senior) może łatwo kliknąć, wykorzystuje się kontrolkę `Button`. W projekcie CareLink przyciski zostały zaprojektowane z myślą o dostępności (*Accessibility*) – posiadają dużą wysokość oraz zaokrąglone rogi, przykład pokazany na Listing 1 (s. 12)

```
<Button Text="ROZPOCZNIJ SPACER"
        Command="{Binding ToggleWalkCommand}"
        BackgroundColor="#4CAF50"
        TextColor="White"
        FontAttributes="Bold"
        FontSize="16"
        CornerRadius="12"
        HeightRequest="55" />
```

Listing 1. Przycisk rozpocznij spacer

  - `Text`: Etykieta tekstowa wyświetlana na przycisku.
  - `Command`: Wiązanie (*Binding*) łączące akcję kliknięcia przycisku z odpowiednią metodą w klasie ViewModel (`ToggleWalkCommand`).
  - `BackgroundColor` / `TextColor`: Kolor tła (np. zielony `#4CAF50` dla akcji pozytywnych / startu) oraz kolory napisu.
  - `HeightRequest="55"`: Zapewnia odpowiednio dużą powierzchnię dotykową, co jest kluczowe dla wygody osób starszych.

- Jak dodać kafelki informacyjne i karty? (`Border`)

  W nowocześniejszym .NET MAUI do tworzenia estetycznych paneli, ramek i kafelków wykorzystuje się kontrolkę `Border` (która zastąpiła przestarzały element `Frame`).

  Przykład kafelka statystyk z zaokrąglonymi rogami, w którym wyświetlany jest przebyty dystans na Listing 2 (s. 13)

```
<Border BackgroundColor="White" StrokeShape="RoundRectangle 16"
    StrokeThickness="0" Padding="20">
    <VerticalStackLayout Spacing="5">
        <Label Text="Przebyty dystans" FontSize="14" TextColor=
   "#666666" HorizontalTextAlignment="Center" />
        <Label Text="{Binding DistanceText, TargetNullValue
   ='0.00 km'}" FontSize="36" FontAttributes="Bold" TextColor=
   "#1976D2" HorizontalTextAlignment="Center" />
    </VerticalStackLayout>
</Border>
```

Listing 2. Kafelek z informacjami o przebytym dystansie

  - `StrokeShape="RoundRectangle 16"`: Odpowiada za zaokrąglenie rogów ramki o promień 16 pikseli.
  - `StrokeThickness="0"`: Usuwa widoczną obramówkę, nadając kafelkowi nowoczesny, płaski wygląd.
  - **Wewnątrz** `Border`: Umieszczono pionowy stos (`VerticalStackLayout`), który grupuje etykietę opisu oraz powiązaną dynamicznie wartość numeryczną (`DistanceText`).

3.4. Struktura layoutów i rozmieszczenie elementów

W projekcie *CareLink* do zarządzania układem graficznym poszczególnych ekranów wykorzystano zaawansowane kontenery rozmieszczenia (*Layouts*) dostarczane przez framework .NET MAUI. Zapewniają one elastyczność oraz responsywność interfejsu na różnych rozdzielczościach ekranów urządzeń mobilnych.

Główne rodzaje layoutów zastosowane w aplikacji to:

- `Grid` **(Siatka układu wierszowego i kolumnowego):** Jest to główny kontener strukturalny wykorzystany na ekranach takich jak `CompassPage`, `WalkPage` oraz `MainPage`. Siatka pozwala na precyzyjny podział przestrzeni za pomocą

właściwości `RowDefinitions` oraz `ColumnDefinitions`, dzieląc ekran na logiczne strefy:

  - **Nagłówek (wiersz typu `Auto`):** Dopasowuje swoją wysokość automatycznie do zawartego tekstu tytułu i statusu (np. zielony pasek statusu opiekuna i baterii na `MainPage`).
  - **Obszar główny (wiersz typu `*` lub umieszczony w `ScrollView`):** Zajmuje całą pozostałą, elastyczną przestrzeń ekranu, centrując kluczowe elementy interaktywne (przycisk SOS, tarczę kompasu lub kafelki statystyk).
  - **Panel dolny (wiersz typu `Auto`):** Skupia przyciski akcji na dole ekranu, zapewniając ich łatwą dostępność pod kciukiem użytkownika.

- `VerticalStackLayout` **(Pionowy stos elementów):** Kontener służący do automatycznego układania elementów jeden pod drugim w kolumnie z zachowaniem stałych odstępów (*Spacing*). Wykorzystywany jest na ekranach logowania i rejestracji (`LoginPage`, `RegisterPage`), a także wewnątrz kafelków `Border` oraz paneli przycisków.

- `ScrollView` **(Widok ze przewijaniem):** Kontener otaczający stos elementów na ekranie głównym `MainPage`, zapobiegający obcinaniu treści na urządzeniach z mniejszymi wyświetlaczami.

Dzięki połączeniu kontrolki `Grid` z elastycznymi kontenerami *StackLayout*, interfejs pozostaje czytelny, estetyczny i w pełni dostosowany do potrzeb osób starszych, unikając nakładania się poszczególnych elementów graficznych.

3.5. Dokładne określenie założeń i działania ekranów

Przepływ użytkownika w aplikacji CareLink został podzielony na odrębne, wyspecjalizowane ekrany:

1. **Ekran Inicjalizacji (`LoadingPage`):**
   - Brak paska nawigacyjnego (`Shell.NavBarIsVisible="False"`).
   - Zawiera wskaźnik ładowania (`ActivityIndicator`) oraz etykietę tekstową.
   - Po automatycznym odczekaniu symulacji inicjalizacji (2 sekundy) następuje nawigacja do ekranu logowania (`LoginPage`).
2. **Ekran Logowania (`LoginPage`):**

- Użytkownik wprowadza nazwę użytkownika (`UsernameEntry`) oraz hasło (`PasswordEntry`).
- Kliknięcie przycisku "Zaloguj się"wywołuje zdarzenie `OnLoginClicked`. Weryfikowane jest wbudowane konto administratora (`admin`/`admin`) lub wpis w lokalnej bazie danych poprzez `LocalDatabaseService`. Po pomyślnym zalogowaniu następuje przejście do ekranu głównego (`MainPage`).
- Kliknięcie "Nie masz konta? Zarejestruj się"otwiera ekran rejestracji.

3. **Ekran Rejestracji (`RegisterPage`):**
   - Użytkownik wypełnia pola tekstowe oraz dokonuje wyboru roli ("Senior"lub "Opiekun") za pomocą kontrolki `Picker`.
   - Kliknięcie "Zarejestruj się"zapisuje użytkownika w pamięci urządzenia za pomocą SQLite i przekierowuje z powrotem do logowania.
4. **Pulpit Główny Seniora (`MainPage`):**
   - **Pasek stanu:** Wyświetla informację o połączeniu z opiekunem, aktualny poziom naładowania baterii urządzenia oraz zegar cyfrowy odświeżany co sekundę. Zawiera także przycisk wylogowania.
   - **Przycisk SOS:** Centralny, duży okrągły przycisk o wzmocnionym czerwonym obramowaniu. Kliknięcie uruchamia akcję `TriggerSosCommand`, wyświetlając powiadomienie o wysłaniu sygnału ratunkowego do opiekuna.
   - **Kafelki Narzędziowe:** Interaktywne ramki `Border` z rozpoznawaniem gestów dotykowych (`TapGestureRecognizer`):
     - *Kompas do domu*: Otwiera widok `CompassPage`.
     - *Mój Spacer*: Otwiera widok `WalkPage` i prezentuje dzisiejszy dystans.
     - *Kontakt z opiekunem*: Wywołuje bezpośrednie połączenie telefoniczne z opiekunem.
5. **Ekran Kompasu (`CompassPage`):**
   - Prezentuje okrągłą tarczę kompasu z umieszczoną wewnątrz etykietą strzałki (emoji). Strzałka wykonuje dynamiczny obrót (`Rotation`) wyznaczony przez właściwość `ArrowRotation`.
   - Przyciski umożliwiają przełączanie trybu szukania (np. Północ magnetyczna / Szukanie domu) oraz powrót do ekranu głównego z wyłączeniem czujnika.

6. **Ekran Spaceru (`WalkPage`):**
    - Wyświetla w formie dużych kafelków aktualny status sygnału GPS, przebyty dystans oraz czas trwania.
    - Przycisk akcji przełącza stan rejestrowania (Start / Stop) i aktualizuje status tekstowy.

3.6. Opis wykorzystanych algorytmów i logiki urządzeń

W aplikacji CareLink zaimplementowano kilka kluczowych algorytmów oraz mechanizmów sprzętowych:

- **Algorytm aktualizacji tarczy kompasu w czasie rzeczywistym:** Klasa `CompassPageViewModel` subskrybuje zdarzenie `ReadingChanged` czujnika Compass.Default. Odebrany azymut magnetyczny h (`HeadingMagneticNorth`) przeliczany jest na obrót strzałki w interfejsie użytkownika według zależności:

  $$\text{ArrowRotation} = -h$$

  Znak ujemny zapewnia, że przy obrocie urządzenia w prawo strzałka wizualnie zachowuje stały kierunek północny.

- **Pętla odświeżania zegara i kontroli baterii:** W modelu `MainPageViewModel` wykorzystano zegar systemowy `IDispatcherTimer` skonfigurowany na interwał 1 sekundy. W każdej tyknięciu pętli aktualizowana jest godzina, natomiast raz na minutę (gdy sekunda wynosi 0) następuje odczyt stanu baterii z API systemowego `Battery.Default.ChargeLevel` i przeliczenie go na wartość procentową.

- **Asynchroniczny dostęp do bazy danych (SQLite):** Dostęp do bazy lokalnej realizowany jest przy użyciu wzorca *Lazy Initialization*. Połączenie SQLiteAsyncConnection inicjalizowane jest tylko raz przed wykonaniem zapytania za pomocą metody `InitAsync()`, co zapobiega wyciekom pamięci i zablokowaniu wątku interfejsu (UI Thread).

4. Implementacja

4.1. Opis nowo utworzonych plików za co odpowiadają

W strukturze projektu wyróżnić można następujące pliki źródłowe, z których każdy pełni określoną rolę w architekturze MVVM i .NET MAUI:

- `App.xaml` **oraz** `App.xaml.cs`: Główny punkt startowy aplikacji. Odpowiada za inicjalizację zasobów globalnych (arkusze stylów, kolory) oraz definicję głównego okna aplikacji (`Window`), osadzając w nim główny kontener nawigacyjny `AppShell`.
- `AppShell.xaml` **oraz** `AppShell.xaml.cs`: Definiuje szkielet nawigacji opartej na trasach (*Shell Routing*). W pliku kodu z użyciem `Routing.RegisterRoute` rejestrowane są ścieżki poszczególnych stron (`LoadingPage`, `LoginPage`, `RegisterPage`, `MainPage`, `CompassPage`, `WalkPage`), co umożliwia nawigację asynchroniczną za pomocą URI.
- `MauiProgram.cs`: Klasa konfiguracyjna odpowiedzialna za budowanie aplikacji (`MauiAppBuilder`). Rejestruje czcionki systemowe oraz konfiguruje logowanie debugowania dla środowiska deweloperskiego.
- `User.cs` (katalog `Models`): Definiuje strukturę danych użytkownika w lokalnej bazie SQLite. Zawiera atrybuty mapowania tabeli (`[PrimaryKey]`, `[AutoIncrement]`).
- **Serwisy (**`Services/`**)**:
  - `LocalDatabaseService.cs`: Zarządza lokalną bazą danych SQLite. Tworzy asynchronicznie tablice oraz realizuje operacje CRUD dla użytkowników.
  - `DatabaseService.cs`: Odpowiada za komunikację z zewnętrznym serwerem bazodanowym w technologii Firebird przy użyciu biblioteki `FirebirdClient`.
- **Modele Widoków (**`PageModels/`**)**: Zawierają logikę biznesową widoków opartą o wzorzec MVVM (`ObservableObject`):
  - `MainPageViewModel.cs`: Obsługuje timer odświeżania czasu, odczyt poziomu baterii, wywołanie alarmu SOS oraz akcje nawigacyjne pulpitu.
  - `CompassPageViewModel.cs`: Zarządza czujnikiem kompasu, przelicza azymut na obrót strzałki oraz obsługuje tryby pracy.

  - `WalkPageViewModel.cs`: Steruje stanem rejestrowania spaceru oraz formatowaniem tekstowym dystansu i czasu.

- **Widoki (`Pages/`)**: Pliki interfejsu użytkownika XAML wraz z plikami kodu w C (`.xaml.cs`), które inicjują komponenty widoku (`InitializeComponent`) oraz wiążą je z odpowiednimi modelami widoków (`BindingContext`).

4.2. Szkielet kodu, struktury i zmienne

W aplikacjach .NET MAUI opartych na MVVM struktura danych jest hermetyzowana w modelach oraz klasach widoków. W Listing 3 (s. 18) przedstawiono szkielet modelu użytkownika (`User.cs`), który definiuje strukturę bazy danych SQLite:

```
using SQLite;

namespace AplikacjaMobilna.Models;

public class User
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
}
```

Listing 3. Definicja struktury bazy danych

- **Struktura i zmienne klasy `User`:**

  - `Id` (typ `int`): Klucz główny tabeli, inkrementowany automatycznie przez silnik SQLite.

  - `Username` (typ `string`): Zmienna przechowująca nazwę użytkownika (login). Inicjalizowana domyślnie jako pusty ciąg znaków.

  - `Password` (typ `string`): Przechowuje hasło użytkownika.

  - `Role` (typ `string`): Określa uprawnienia użytkownika w systemie (np. *Senior* lub *Opiekun*).

4.3. Opis dziedziczenia w architekturze aplikacji

Mechanizm dziedziczenia odgrywa kluczową rolę w strukturze klas frameworka .NET MAUI oraz biblioteki MVVM Community Toolkit:

- **Dziedziczenie po klasach bazowych UI (`ContentPage`, `Application`, `ContentPage`, `Shell`):**
  - Wszystkie ekrany aplikacji (np. `LoginPage`, `MainPage`, `CompassPage`) dziedziczą po klasie `ContentPage` dostarczanej przez bibliotekę Microsoft.Maui.Controls. Dzięki temu dziedziczą pełen cykl życia strony (np. metodę `OnAppearing()`) oraz właściwości układu wizualnego.
  - Klasa `App` dziedziczy po `Application`, a `AppShell` dziedziczy po klasie `Shell`, co pozwala na obsługę globalnego okna oraz routingu.
- **Dziedziczenie i implementacja interfejsów w logice ViewModel**
  - Modele widoków dziedziczą po klasie bazowej `ObservableObject` z pakietu *CommunityToolkit.Mvvm*.
  - Klasa ta implementuje interfejs `INotifyPropertyChanged`, co pozwala na automatyczne powiadamianie widoku XAML o zmianach wartości właściwości (mechanizm *Data Binding*).
  - Wybrane klasy (np. `MainPageViewModel`) implementują dodatkowo interfejs `IDisposable` w celu zwolnienia zasobów systemowych (np. zatrzymania timera przy niszczeniu obiektu).

4.4. Opis procedur, metod i ich działania

Logika biznesowa i operacje asynchroniczne w aplikacji zostały zamknięte w metodach serwisów oraz klas ViewModel. Poniżej omówiono wybrane kluczowe procedury wraz z prezentacją ich kodu źródłowego.

- W procesie rejestracji użytkowników kluczową rolę odgrywa asynchroniczna metoda sprawdzająca istnienie konta oraz zapisująca dane w lokalnej bazie danych SQLite. Jej implementację zaprezentowano na Listing 4 (s. 19).

```
1 public async Task<bool> RegisterUserAsync(string username,
    string password, string role)
2 {
3     await InitAsync();
4
```

```
    var existingUser = await _database!.Table<User>()
        .Where(u => u.Username == username)
        .FirstOrDefaultAsync();

    if (existingUser != null) return false;

    var newUser = new User { Username = username, Password =
    password, Role = role };
    int result = await _database.InsertAsync(newUser);
    return result > 0;
}
```

Listing 4. Rejestracja użytkownika w bazie lokalnej

Metoda `RegisterUserAsync` najpierw inicjalizuje połączenie bazodanowe (`InitAsync()`), a następnie asynchronicznie sprawdza w tabeli, czy użytkownik o podanej nazwie już istnieje. Jeśli zmienna `existingUser` nie jest pusta, operacja zostaje przerwana i zwracana jest wartość `false`. W przeciwnym wypadku tworzony jest nowy obiekt encji `User` i wykonywany jest asynchroniczny zapis (`InsertAsync`), którego rezultat decyduje o powodzeniu operacji.

- Do obsługi dynamicznego obrotu tarczy kompasu na ekranie nawigacyjnym wykorzystano procedurę zdarzeniową czujnika urządzeń mobilnych. Kod odpowiedzialny za tę logikę przedstawiono na Listing 5 (s. 20).

```
private void Compass_ReadingChanged(object sender,
    CompassChangedEventArgs e)
{
    double heading = e.Reading.HeadingMagneticNorth;
    ArrowRotation = -heading;
}
```

Listing 5. Czujnik kompasu

Procedura `Compass_ReadingChanged` nasłuchuje cyklicznych zmian odczytów czujnika magnetycznego (`Compass.Default`). Pobiera aktualny kierunek północy magnetycznej (`HeadingMagneticNorth`) i przypisuje jego negatywną wartość do właściwości `ArrowRotation`, co bezpośrednio steruje płynnym obrotem graficznym elementu strzałki w interfejsie XAML.

- Za cykliczny pomiar czasu oraz kontrolowanie stanu baterii urządzenia na pulpicie głównym odpowiada mechanizm timera. Jego konfigurację zaprezentowano na Listing 6 (s. 20).

```
_timer = Application.Current?.Dispatcher.CreateTimer();
```

```
if (_timer != null)
{
    _timer.Interval = TimeSpan.FromSeconds(1);
    _timer.Tick += (s, e) =>
    {
        UpdateCurrentTime();
        if (DateTime.Now.Second == 0) UpdateBatteryLevel();
    };
    _timer.Start();
}
```

Listing 6. Inicjalizacja pętli czasowej odświeżania zegara i baterii

W konstruktorze modelu widoku tworzony jest asynchroniczny zegar interwałowy o częstotliwości jednej sekundy. W każdym cyklu wywoływana jest metoda odświeżająca zegar systemowy, natomiast raz na minutę (w momencie, gdy liczba sekund wynosi zero) uruchamiana jest procedura odczytująca aktualny poziom naładowania baterii z poziomu API systemowego.

5. Testowanie

6. Podręcznik użytkownika

Spis rysunków

Spis tabel

Spis listingów

1. Przycisk rozpocznij spacer 12
2. Kafelek z informacjami o przebytym dystansie 13
3. Definicja struktury bazy danych 18
4. Rejestracja użytkownika w bazie lokalnej 19
5. Czujnik kompasu 20
6. Inicjalizacja pętli czasowej odświeżania zegara i baterii 20